Universidad Abierta Interamericana
Facultad de Tecnología Informática
Ingeniería en Sistemas Informáticos

TRABAJO FINAL DE INGENIERÍA

Patrones de diseño y principios SOLID

Dónde está aplicado cada uno, con su archivo

Alumno: Pettini, Patricio Ezequiel
Legajo: B00072710-T1
Sede: Sede Norte — Boulogne
Docentes: Romero, Martín Miguel · Weingand, Gastón Matías

Versión 1.0 · 23/10/2026

Índice

1 · Patrones de diseño 3
2 · Principios SOLID 4
La violación que hay que declarar 5
3 · Seguridad 5
Sobre JWT 6
Sobre Keycloak 6
4 · Testing y calidad 6
5 · Docker 7
6 · Integración continua 8
Resumen 8
Fuentes 10

Verificación al **09/09/2026** contra el código, no contra la memoria. Cada fila apunta al archivo donde vive, para que en la defensa la respuesta a «¿dónde está el patrón X?» sea abrir un archivo y no explicar una intención.

Lo que no está aplicado figura igual, marcado. Una tabla donde todo da verde no es una verificación.

1 · Patrones de diseño

Patrón	Dónde	Qué problema resuelve acá
DAO genérico	`PredictIT.DAO/Contracts/Contratos.cs:14` — `IGenericDao<T>`, heredado por `IEquipoDao`, `IIncidenciaDao`, `IMantenimientoDao`	El CRUD que se repite en toda entidad de negocio
Object Mapper	`Contratos.cs:8` — `IObjectMapper<out T>`, 26 implementaciones: 9 en `Mappers/Mappers.cs`, 13 en `Mappers/MappersIncidencia.cs` y 4 en los DAO de facturación y agenda	Traducir `IDataRecord` a entidad en un solo lugar por tabla
Abstract Factory	`DAO/Factory/FactoryDao.cs`, `BLL/FactoryBusiness.cs`	La API pide contratos y nunca nombra una clase concreta. Es lo que hace sustituible el acceso a datos
Unit of Work	`FactoryDao.cs:142` — `SqlTransactRepository`	El número correlativo de incidencia y la incidencia van juntos o no van: dos altas simultáneas no pueden quedarse con el mismo número
Composite	`Domain/Seguridad/ComponenteSeguridad.cs:11`, con `Familia` y `Patente` como nodo y hoja	Familias que contienen familias, con guarda de ciclos. Es el modelo de permisos de la cátedra
Strategy	`Service/Prediccion/MotorPredictivo.cs:85` — `IEstrategiaRegla` con **5** implementaciones	Cada tipo de regla predictiva se evalúa distinto; agregar una sexta no toca el motor
Adapter	`Service/IA/ProveedorIA.cs` — `IProveedorIA`, con `ProveedorIAClaude` y `ProveedorIASimulado`	Dos proveedores intercambiables por configuración. Es lo que permite demostrar el sistema sin crédito de API
Decorator	`Service/IA/Disyuntor.cs:147` — `ProveedorIAConDisyuntor` envuelve a otro `IProveedorIA`	Agregar resiliencia sin que ningún proveedor sepa que existe el disyuntor

Patrón	Dónde	Qué problema resuelve acá
Circuit Breaker	`Service/IA/Disyuntor.cs:30`	Dejar de llamar a un servicio caído. Es el CP-14
Bulkhead	`DAO/Helpers/SqlHelper.cs` — `ConexionesSql.ParaSegundoPlano()`	El barrido predictivo tiene su propio pool acotado: no puede quedarse con las conexiones que atienden a los usuarios

Los diez están en uso, no declarados. El caso que conviene mirar es `IGenericDao<T>`: tres interfaces lo heredan de verdad. Un patrón declarado y no usado promete una estructura que el sistema no tiene, y en ese caso conviene no declararlo.

Corregido en esta verificación: el bulkhead estaba comprometido en el

ADR 0009 y **no estaba en el código**. La ausencia se volvió concreta al construir el barrido predictivo, que es el proceso que la decisión buscaba contener. Está implementado y probado.

2 · Principios SOLID

Principio	Cómo se sostiene	Estado
Responsabilidad única	El motor predictivo no lee el reloj ni la base: recibe el contexto y devuelve el resultado. Por eso se prueba sin infraestructura	OK
Abierto/cerrado	Una regla predictiva nueva es una clase nueva; el motor no cambia	OK
Sustitución de Liskov	`ProveedorIASimulado` y `ProveedorIAClaude` son intercambiables, y `ProveedorIAConDisyuntor` envuelve a cualquiera de los dos sin que el llamador note la diferencia	OK
Segregación de interfaces	45 interfaces, casi todas acotadas… salvo una	Parcial
Inversión de dependencias	Ninguna capa nombra una clase concreta de la capa de abajo. La `Api` depende de `IFactoryBusiness`; el `Domain` no depende de nadie	OK

La violación que hay que declarar

`IAdministracionBusiness` tiene **18 métodos de cinco temas distintos**: mantenimientos, usuarios, bitácora y errores, integración de IA, y respaldos. Todo consumidor depende de las cinco cosas aunque use una.

El caso incumple de manera directa el principio de segregación de interfaces, y está en el código propio. Se parte en cuatro interfaces en el Sprint 4 del plan. Se declara acá y no se disimula, porque una tabla SOLID con cinco tildes verdes en un sistema de este tamaño no es creíble.

3 · Seguridad

Control	Dónde	Estado
Contraseñas	PBKDF2-HMAC-SHA256, 600 000 iteraciones —la recomendación de OWASP—, comparación en tiempo constante	OK
Cifrado reversible	AES-256-GCM, `Service/Seguridad/CifradoService.cs` (ADR 0010)	OK
Autenticación	JWT HS256, vigencia acotada, clave fuera del repositorio	OK
Autorización	`[RequierePatente]`, patentes leídas de la base **en cada petición** y no del token: revocar un permiso tiene efecto inmediato	OK
Aislamiento multi-organización	Aplicado en el DAO desde el contexto de sesión. El identificador de organización **no viaja en la petición**, así que ningún controlador puede pedir datos de otra	OK
Bitácora inmutable	Disparador `INSTEAD OF UPDATE, DELETE` que rechaza con error 50001	OK
Bloqueo por intentos fallidos	Por usuario, con desbloqueo administrativo	OK
Inyección SQL	ADO.NET con `SqlParameter` en todas las consultas	OK
CORS	Lista blanca de orígenes por configuración	OK
Límite de intentos **por IP**	—	NO S4
Cabeceras de seguridad (HSTS, CSP, `X-Content-Type-Options`)	—	NO S4
Revisión contra OWASP Top 10	—	NO S4

El hueco que importa es el límite por IP. El bloqueo por usuario no frena a quien prueba una contraseña contra cien usuarios distintos, que es como se hace el ataque en la práctica.

Sobre JWT

Es la elección correcta acá y conviene poder justificarla en dos frases: la API es sin estado y el frontend es una SPA en otro origen, así que una cookie de sesión obligaría a manejar CSRF y afinidad de servidor sin comprar nada.

La decisión fina no es usar JWT sino **qué se pone adentro**: el token lleva el usuario y la organización activa, y **no lleva las patentes**. Si las llevara, revocar un permiso no tendría efecto hasta que el token expirara. El costo es una consulta por petición y se paga contento.

Sobre Keycloak

Recomendación: no, y documentarlo.

Keycloak es un IAM serio y resolvería federación, OIDC, MFA y rotación de claves —cosas que hoy no están—. El problema es de contexto: el modelo Familia/Patente con Composite, el hash de contraseñas y la bitácora de eventos de seguridad son **objeto de evaluación** en este trabajo. Delegarlos en un producto externo significa que lo que se evalúa pasa a ser configuración de un contenedor ajeno.

Habría además una desalineación concreta: el modelo de permisos de Keycloak — roles y grupos— no expresa el Composite anidado de familias con guarda de ciclos, así que habría que mantener el modelo propio igual, sincronizado con el externo. Con dos fuentes de verdad para los permisos, cualquier diferencia entre ellas se resuelve por accidente.

Lo que sí corresponde es tratarlo como decisión y no como ausencia: un ADR en el Sprint 4 con el análisis y la ruta de migración para un despliegue real. Eso demuestra el criterio sin sacrificar lo que se evalúa. Si preferís implementarlo, la forma que no rompe nada es **agregarlo como proveedor de autenticación alternativo** detrás de la interfaz actual, dejando el modelo de patentes intacto: Keycloak dice quién sos, PredictIT sigue diciendo qué podés hacer.

4 · Testing y calidad

Ítem	Estado
Pruebas backend (xUnit + FluentAssertions)	**403**, en verde
Pruebas frontend (Vitest)	**95**, en verde
Pruebas de integración contra SQL Server real	OK
Pruebas que verifican propiedades, no sólo casos	OK — la que verifica por reflexión que cada código de bitácora exista en la base, y la que rechaza literales sueltos en las llamadas
`coverlet` instalado	OK
Informe de cobertura publicado	OK — el CI lo sube como artefacto en cada corrida

Ítem	Estado
CP-01..CP-14 con trazabilidad a las pruebas	OK — `docs/tecnico/CASOS-DE-PRUEBA.md` se genera del `.trx`, y hoy no queda ningún caso sin cubrir. Lo que no hay es un `Trait` en el código que los agrupe: el mapeo vive en el generador
Pruebas de carga del RNF del ADR 0008	NO S2

Lo que las pruebas no encontraron, y conviene tenerlo presente: los dos defectos más serios hasta hoy fueron **ausencias**. La bitácora sin eventos de negocio apareció contando filas en la base; el motor predictivo que no se disparaba solo lo reportó el usuario. Ninguna prueba podía fallar, porque no había código roto: faltaba código. Por eso la Definición de Terminado exige verificar sobre el sistema corriendo.

El tercero apareció en una captura de pantalla. El sistema sella fechas con dos relojes —trece columnas del esquema con `DEFAULT SYSDATETIME()`, que es el del motor, y el resto con `DateTime.Now`, que es el de la aplicación—, y el contenedor de la API declaraba su zona horaria mientras el del motor corría en UTC. Una incidencia registrada y asignada en el mismo segundo quedaba con **tres horas** entre las dos marcas, y eso se vio recién cuando la pantalla del solicitante puso las dos una debajo de la otra: «recibimos tu pedido 15:56», «se lo asignamos 18:56». Ninguna prueba fallaba porque cada marca, por separado, era correcta. La red que quedó es `RelojUnicoTests`, que compara los dos relojes y falla si la diferencia llega a un minuto: menos que la distancia entre dos zonas horarias, que es de media hora como mínimo.

5 · Docker

Ítem	Estado
SQL Server 2022 con volumen persistente y healthcheck	OK
Volumen de respaldos mapeado al host	OK
Imagen de la API	OK — `backend/Dockerfile`
Imagen del frontend	OK — `frontend/Dockerfile`, con Nginx sirviendo el build
`.dockerignore`	OK — uno por imagen
Un `docker compose up` que levante todo	OK — cuatro servicios: `sqlserver`, `migraciones`, `api` y `web`

Ya no hace falta tener .NET 9 ni Node instalados: `docker compose up --build` levanta las cuatro piezas y el servicio `migraciones` aplica el esquema y los datos de demostración antes de que arranque la API. Es la instrucción única del LEEME de la entrega, y es también la que corre el CI para comprobar que las dos imágenes construyen.

6 · Integración continua

`.github/workflows/ci.yml`, sobre cada `push` y cada `pull request` a `main`. Cinco trabajos independientes, y separados a propósito: con un solo trabajo el primer fallo esconde todo lo demás.

Trabajo	Qué verifica
Backend	Compila y corre las 403 pruebas contra un SQL Server 2022 real, con el esquema y los datos aplicados desde los scripts
Documentación	Los trece controles de `docs/_verificar-alineacion.py`: que el documento, el Excel y el código digan lo mismo
Frontend	Tipos (`tsc --noEmit`), las 34 pruebas, el build y el control de textos fuera del diccionario de traducciones
Dependencias	Paquetes de .NET y de npm con vulnerabilidades conocidas
Docker	Que las dos imágenes construyan

El criterio de aceptación de la historia H-02 pedía **verificar que el CI falle** cuando algo se rompe, porque un pipeline que nunca se puso en rojo no se sabe si funciona. Se cumplió sin buscarlo: el control de claves únicas del seed lo puso en rojo con un identificador duplicado, y ese es el registro de que el pipeline hace lo que dice.

Resumen

Área	Estado	Detalle
Patrones de diseño	OK	10 de 10, todos en uso
SOLID	Parcial	4 de 5; una violación declarada, con fecha de corrección
Seguridad	Parcial	Base sólida; faltan límite por IP, cabeceras y revisión OWASP
Testing	Parcial	498 pruebas en verde (403 backend, 95 frontend)
Docker	OK	Las cuatro piezas, con un solo `docker compose up --build`
RBAC	OK	Completo, con patentes verificadas por petición
JWT	OK	Aplicado, y con criterio sobre qué contiene
CI/CD	OK	Cinco trabajos sobre cada push a `main`
Keycloak	Decisión pendiente de documentar (recomendación: no adoptarlo)	

Fuentes

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1995). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2.ª ed.). Pragmatic Bookshelf.